0x Guard

Smart contracts security assessment

Final report

Tariff: Standard

Curveball

October 2026

0xguard.com

hello@0xguard.com

Contents

1. Introduction 3
2. Contracts checked 4
3. Procedure 4
4. Known vulnerabilities checked 5
5. Classification of issue severity 6
6. Issues 6
7. Conclusion 9
8. Disclaimer 10

Introduction

The report has been prepared for **Curveball**.

The code is available at the stagezerolabs/curveball-contracts GitHub repository and was audited after the commit 98aa5a0b71a02332bf2aa1814fc9019bf2912525. The test suit includes unit-, integration-, fuzzing-, and fork-testing.

The CurveLaunchpad acts as a platform for deploying and managing fixed-supply tokens using a virtual bonding curve mechanism, eventually transitioning successful tokens to a decentralized exchange (Icarus Finance for RISE network or other ve(3,3) Velodrome-like project).

The launchpad allows creators to deploy new tokens at deterministic addresses by providing metadata and optionally allocating a portion of the supply to a dedicated reserve, that is stored into the CurveHolderReserve contract. While a token remains on the launchpad, users can buy or sell it directly through the contract using virtual reserves. Built-in trading fees and taxes are automatically calculated and routed to an escrow distribution system during these transactions. Once the predefined curve supply is fully sold out, the token reaches a state ready for graduation. This permissionless process enables anyone to trigger the migration, which automatically provisions a new liquidity pool on the Icarus exchange at the final curve price.

Locked liquidity is stored in the CurveLpLocker contract that allows to claim fees from liquidity pools and redirects them to the CurveFeeEscrow contract. The CurveFeeEscrow accrues fees and taxes from liquidity pools and launchpad trades and distributes them according to split scheme between the token creator, the protocol, and the buyback option.

The CurveToken contract is a fixed-supply ERC20 token that strictly blocks all transfers between external addresses prior to the graduation phase of the token market.

Report update. The contracts' code was updated according to this report and rechecked after the commit 20c8ea696d80666ec4004dc2084098ba10ae03f4.

Name	Curveball
Audit date	2026-10-23 - 2026-10-27
Language	Solidity
Platform	RISE Chain

Contracts checked

Name	Address
CurveLaunchpad	
CurveToken	
CurveLpLocker	
CurveHolderReserve	
CurveFeeEscrow	
CurveMath	

Procedure

We perform our audit according to the following procedure:

Automated analysis

- Scanning the project's smart contracts with several publicly available automated Solidity analysis tools
- Manual verification (reject or confirm) all the issues found by the tools

Manual audit

- Manually analyze smart contracts for security vulnerabilities
- Smart contracts' logic check

Known vulnerabilities checked

Title	Check result
Unencrypted Private Data On-Chain	passed
Code With No Effects	passed
Message call with hardcoded gas amount	passed
Typographical Error	passed
DoS With Block Gas Limit	passed
Presence of unused variables	passed
Incorrect Inheritance Order	passed
Requirement Violation	passed
Weak Sources of Randomness from Chain Attributes	passed
Shadowing State Variables	passed
Incorrect Constructor Name	passed
Block values as a proxy for time	passed
Authorization through tx.origin	passed
DoS with Failed Call	passed
Delegatecall to Untrusted Callee	passed
Use of Deprecated Solidity Functions	passed
Assert Violation	passed
State Variable Default Visibility	passed
Reentrancy	passed
Unprotected SELFDESTRUCT Instruction	passed
Unprotected Ether Withdrawal	passed
Unchecked Call Return Value	passed

Floating Pragma	passed
Outdated Compiler Version	passed
Integer Overflow and Underflow	passed
Function Default Visibility	passed

Classification of issue severity

High severity	High severity issues can cause a significant or full loss of funds, change of contract ownership, major interference with contract logic. Such issues require immediate attention.
Medium severity	Medium severity issues do not pose an immediate risk, but can be detrimental to the client's reputation if exploited. Medium severity issues may lead to a contract failure and can be fixed by modifying the contract state or redeployment. Such issues require attention.
Low severity	Low severity issues do not cause significant destruction to the contract's functionality. Such issues are recommended to be taken into consideration.

Issues

High severity issues

No issues were found

Medium severity issues

1. TWAP is not reliable (CurveFeeEscrow)
Status: Fixed

The pool can be created without minting, and the public sync function pushes zeroed cumulative prices observations. This means that the first buyback's price can be manipulated, and the 25% buyback chunk may be lost almost completely. Next buybacks may also be impacted until

observation array will be filled with data.

Recommendation: Possible remediations may be the follows. Check the first observation in window against zero data and skip. Check and store the `pool.observationLength` at the moment of market graduation and use it in buyback as zero point. Skip the buyback if `twapOut` is 0 (affects only the first buyback).

Report update: Seed observation is now stored at the moment of market graduation.

Low severity issues

1. Quote token limitations (CurveLaunchpad)

Status: Fixed

The quoteToken is an arbitrary external payment token. However, both CurveLaunchpad and CurveMath contracts implicitly assume that the quoteToken uses 18 decimals. Integrating a payment token with a different number of decimals will lead to incorrect math calculations and unexpected trading outputs.

Recommendation: Validate the token's decimals in the constructor (e.g., `require(token.decimals() == 18, "Invalid decimals")`) or explicitly document this limitation for supported quote tokens.

Report update: Strict requirement for 18 decimals was included in code and documentation.

2. Transfers to launchpad (CurveToken)

Status: Fixed

Token transfers are prohibited unless market is graduated or transfer is made from or to launchpad. This means that users can send their tokens to the CurveLaunchpad contract while the market is in trading or aborted state. The successful graduation burns those mistakenly sent tokens and the owner can't rescue them.

Recommendation: Safety guard `(to == launchpad && msg.sender == launchpad )` may be added into

`_beforeTokenTransfer`.

Report update: Suggested fix was implemented.

3. Locked reserves (CurveHolderReserve)

Status: Fixed

The CurveHolderReserve contract cannot release reserved tokens from failed markets. This does not impact functionality, as these tokens should only unlock post-graduation. However, it may confuse users tracking large token holders. This limitation is intentional/by design: the system cannot differentiate between temporarily aborted and permanently failed market states.

Report update: Explicit getter `lockedReserveOf` was added.

4. Locked quote tokens (CurveFeeEscrow)

Status: Fixed

The CurveFeeEscrow contract cannot spend buyback part of quote tokens from failed markets. This does not impact functionality, and the total locked amount is not traceable easily. This limitation is intentional/by design: the system cannot differentiate between temporarily aborted and permanently failed market states.

Report update: Explicit getter `lockedBuyback` was added.

Conclusion

Curveball CurveLaunchpad, CurveToken, CurveLpLocker, CurveHolderReserve, CurveFeeEscrow, CurveMath contracts were audited. 1 medium, 4 low severity issues were found.

1 medium, 4 low severity issues have been fixed in the update.

Disclaimer

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability)set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without 0xGuard prior written consent.

This report is not, nor should be considered, an “endorsement” or “disapproval” of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts 0xGuard to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.



Guard